

Insider Threat Detection Using Random Forest

This notebook predicts malicious employee behavior using behavioral and access data.

- Dataset: `insider_threat_clean_dataset.csv` (~118k records)
- Goal: Detect `is_malicious` employees
- Highlights: Random Forest Model with class imbalance handling, ROC-AUC evaluation, feature importance

```
In [1]: # Imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

sns.set_style("whitegrid")
```

Load Dataset & Inspect

We load the CSV dataset and separate the **target variable** (`is_malicious`) from input features.

We also identify the **categorical columns** for one-hot encoding.

```
In [2]: # Load dataset
df = pd.read_csv("insider_threat_clean_dataset.csv")

print("Dataset shape:", df.shape)
print(df.head())
print(df.info())

# Separate features and target
y = df["is_malicious"]
X = df.drop("is_malicious", axis=1)

# Identify categorical columns
categorical_columns = [
    "employee_department",
    "employee_campus",
    "employee_position",
```

```
    "employee_origin_country"  
]
```

Dataset shape: (118614, 22)

	employee_department	employee_campus	employee_position	\
0	Engineering Department	Campus C	Design Engineer	
1	Engineering Department	Campus C	Design Engineer	
2	Engineering Department	Campus C	Design Engineer	
3	Engineering Department	Campus C	Design Engineer	
4	Engineering Department	Campus C	Design Engineer	

	employee_seniority_years	is_contractor	employee_classification	\
0	22	0	2	
1	22	0	2	
2	22	0	2	
3	22	0	2	
4	22	0	2	

	has_foreign_citizenship	has_criminal_record	has_medical_history	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	employee_origin_country	...	total_files_burned	burned_from_other	\
0	Georgia	...	4	0	
1	Georgia	...	0	0	
2	Georgia	...	2	0	
3	Georgia	...	0	0	
4	Georgia	...	0	0	

	is_abroad	trip_day_number	hostility_country_level	num_entries	\
0	0	0.0	0	1	
1	0	0.0	0	1	
2	0	0.0	0	1	
3	0	0.0	0	1	
4	0	0.0	0	0	

	num_unique_campus	late_exit_flag	entry_during_weekend	is_malicious
0	1	0	1	0
1	1	0	0	0
2	1	0	0	0
3	1	0	0	0
4	0	0	0	0

[5 rows x 22 columns]

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 118614 entries, 0 to 118613

Data columns (total 22 columns):

#	Column	Non-Null Count	Dtype
0	employee_department	118614 non-null	object
1	employee_campus	118614 non-null	object
2	employee_position	118614 non-null	object
3	employee_seniority_years	118614 non-null	int64
4	is_contractor	118614 non-null	int64
5	employee_classification	118614 non-null	int64
6	has_foreign_citizenship	118614 non-null	int64

```

7  has_criminal_record      118614 non-null  int64
8  has_medical_history      118614 non-null  int64
9  employee_origin_country  118614 non-null  object
10 total_printed_pages      118614 non-null  int64
11 num_printed_pages_off_hours 118614 non-null  int64
12 total_files_burned       118614 non-null  int64
13 burned_from_other        118614 non-null  int64
14 is_abroad                118614 non-null  int64
15 trip_day_number          118614 non-null  float64
16 hostility_country_level  118614 non-null  int64
17 num_entries              118614 non-null  int64
18 num_unique_campus        118614 non-null  int64
19 late_exit_flag           118614 non-null  int64
20 entry_during_weekend     118614 non-null  int64
21 is_malicious             118614 non-null  int64

```

dtypes: float64(1), int64(17), object(4)

memory usage: 19.9+ MB

None

Preprocessing & Random Forest Pipeline

- One-hot encode categorical features
- Use `RandomForestClassifier` with `class_weight='balanced'` to handle class imbalance
- Build a scikit-learn pipeline

```

In [3]: # Preprocessing
preprocessor = ColumnTransformer(
    transformers=[("cat", OneHotEncoder(handle_unknown="ignore"), categorical_columns),
                 ("passthrough", "passthrough")
    ])

# Random Forest model
model = RandomForestClassifier(
    n_estimators=200,
    max_depth=10,
    random_state=1,
    class_weight='balanced' # Address class imbalance
)

# Build pipeline
pipeline = Pipeline([
    ("preprocess", preprocessor),
    ("model", model)
])

```

Split Data & Train Model

Split the data into **80% training** and **20% testing**, then fit the pipeline.

```
In [4]: # Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1
)

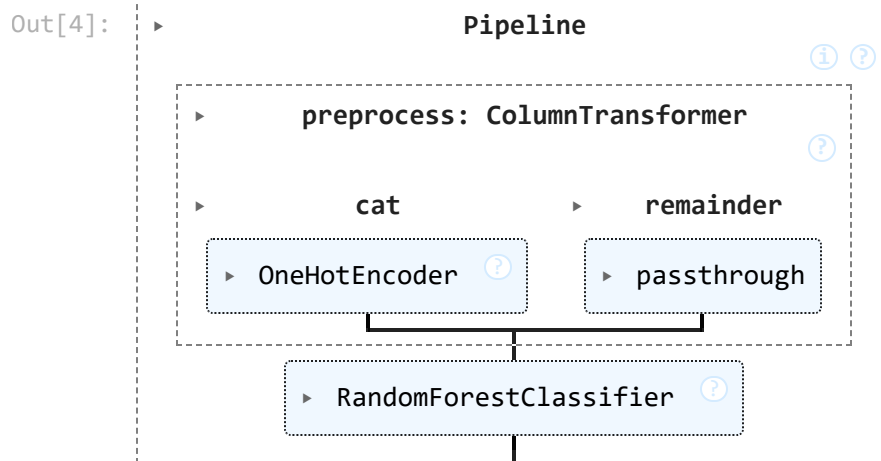
print("Training size:", X_train.shape)
print("Test size:", X_test.shape)

# Train the model
print("\nTraining model...")
pipeline.fit(X_train, y_train)
```

Training size: (94891, 21)

Test size: (23723, 21)

Training model...



Model Evaluation

- Accuracy & classification report
- Confusion matrix
- ROC curve and AUC score

```
In [5]: # Predictions
y_pred = pipeline.predict(X_test)

# Accuracy & classification report
print("\nModel Accuracy:")
print(accuracy_score(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.title("Confusion Matrix")
```

```

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.show()

# ROC Curve + AUC Score
y_probs = pipeline.predict_proba(X_test)[:,-1]
fpr, tpr, thresholds = roc_curve(y_test, y_probs)
roc_auc = auc(fpr, tpr)
print("\nAUC Score:", roc_auc)

plt.figure(figsize=(6,5))
plt.plot(fpr, tpr, label="ROC Curve (AUC = %0.3f)" % roc_auc)
plt.plot([0,1], [0,1], linestyle="--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.tight_layout()
plt.legend()
plt.show()

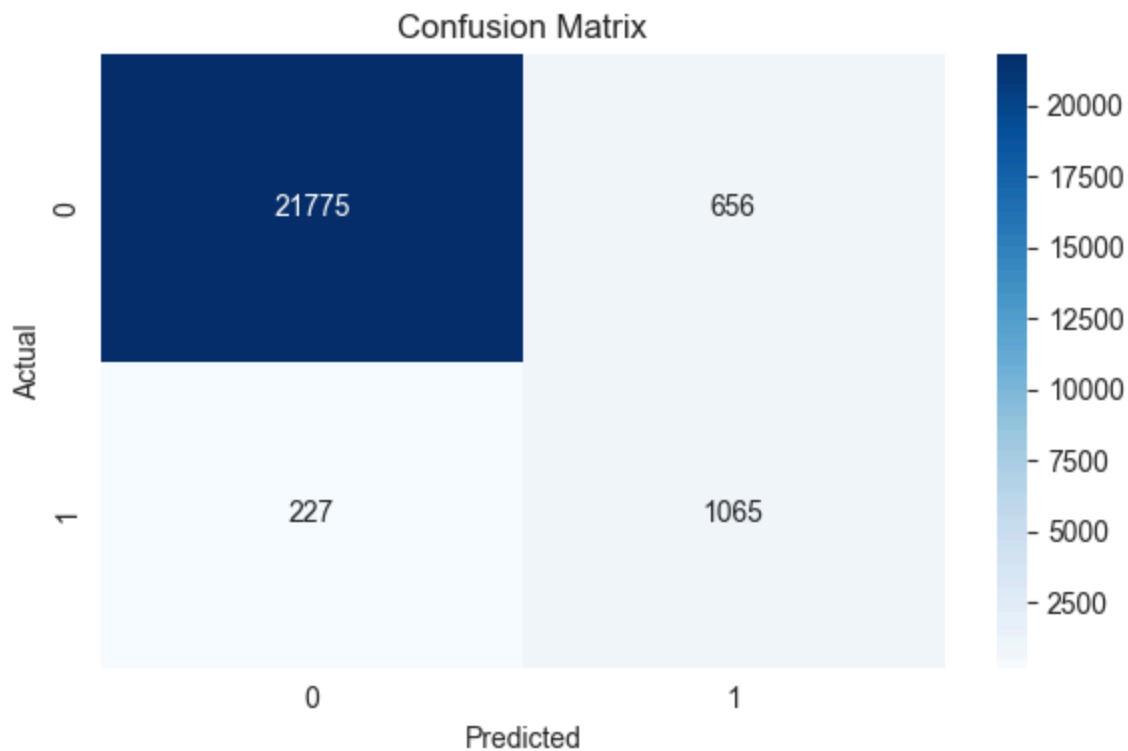
```

Model Accuracy:

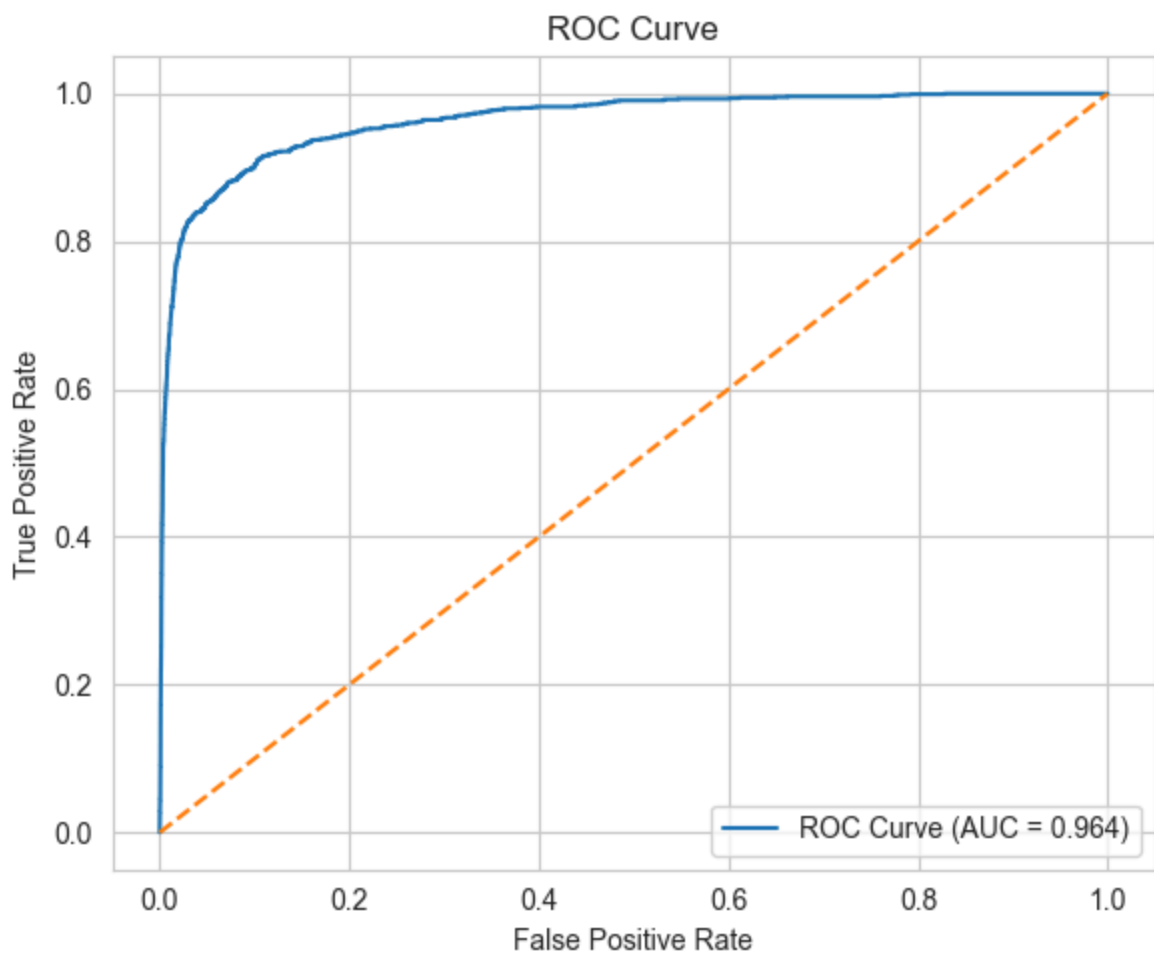
0.9627787379336509

Classification Report:

	precision	recall	f1-score	support
0	0.99	0.97	0.98	22431
1	0.62	0.82	0.71	1292
accuracy			0.96	23723
macro avg	0.80	0.90	0.84	23723
weighted avg	0.97	0.96	0.97	23723



AUC Score: 0.9641106652074963



Feature Importance

- Identify which features contribute most to malicious behavior predictions
- Plot the top 10 features

```
In [6]: # Get feature names after encoding
feature_names = pipeline.named_steps["preprocess"].get_feature_names_out()
importances = pipeline.named_steps["model"].feature_importances_

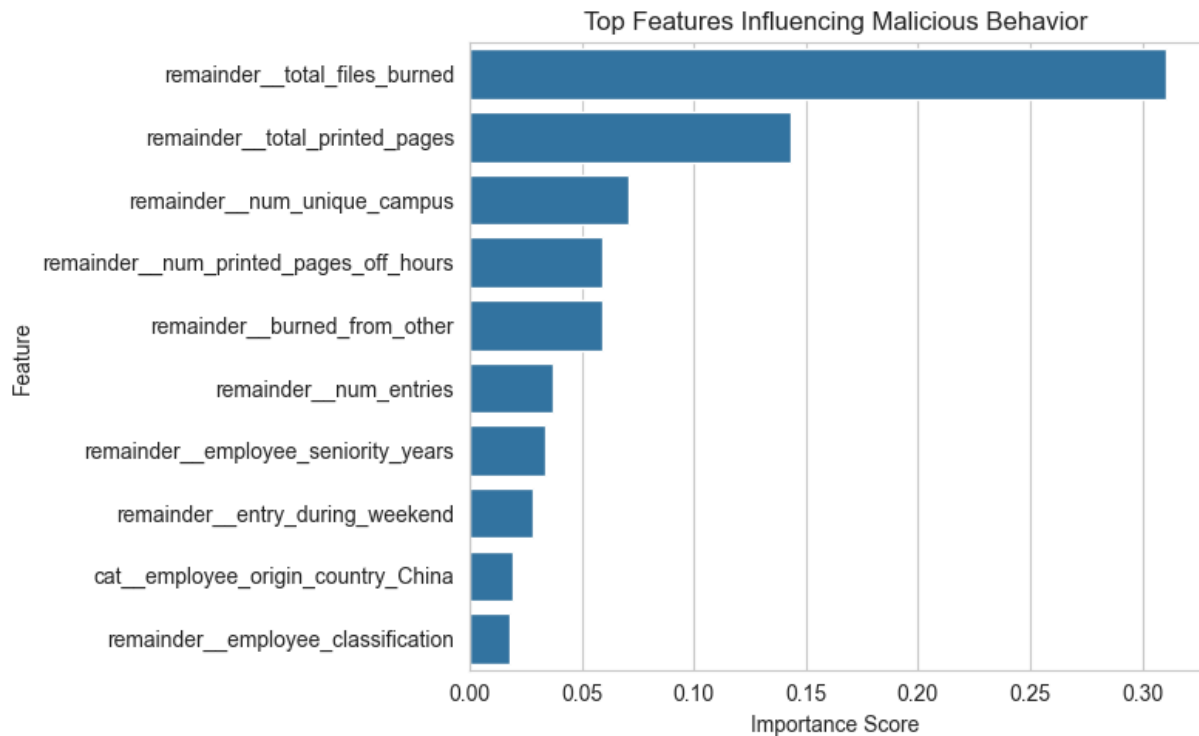
# Create dataframe
feature_importance = pd.DataFrame({
    "feature": feature_names,
    "importance": importances
}).sort_values(by="importance", ascending=False)

print("\nTop 15 Important Features:")
print(feature_importance.head(15))

# Plot top 10 features
top_features = feature_importance.head(10)
plt.figure(figsize=(8,5))
sns.barplot(data=top_features, x="importance", y="feature")
plt.title("Top Features Influencing Malicious Behavior")
plt.xlabel("Importance Score")
plt.ylabel("Feature")
plt.tight_layout()
plt.show()
```

Top 15 Important Features:

	feature	importance
102	remainder__total_files_burned	0.310910
100	remainder__total_printed_pages	0.142965
108	remainder__num_unique_campus	0.071297
101	remainder__num_printed_pages_off_hours	0.059525
103	remainder__burned_from_other	0.059099
107	remainder__num_entries	0.037431
94	remainder__employee_seniority_years	0.034005
110	remainder__entry_during_weekend	0.028407
65	cat__employee_origin_country_China	0.019615
96	remainder__employee_classification	0.017661
0	cat__employee_department_Engineering Department	0.015004
41	cat__employee_position_Integration and Testing...	0.012301
106	remainder__hostility_country_level	0.009852
27	cat__employee_position_Data Scientist	0.008344
7	cat__employee_department_Operations and Manufa...	0.007939



Summary of Findings

Model Performance:

- **Accuracy:** 96% – the model predicts most employees correctly.
- **Malicious Employee Recall:** 82% – it catches most of the risky employees.
- **Precision for Malicious Employees:** 62% – some false alarms, but better to catch threats.
- **AUC Score:** 0.96 – the model ranks high-risk employees well.

Top Features that Affect Predictions:

- `total_files_burned` – the most important indicator.
- `total_printed_pages` – printing a lot can be risky.
- `num_unique_campus` – visiting multiple campuses is suspicious.
- `num_printed_pages_off_hours` – printing during off hours is a red flag.
- `burned_from_other` – handling other employees' files could be risky.
- Other factors: number of entries, seniority, weekend activity, department, and country.

Key Takeaways:

- Employee behavior like off-hours activity, multi-campus access, and file burning helps identify insider threats.
- Using `class_weight='balanced'` improved detection of rare malicious events.
- The model is interpretable and can help security teams spot risky behavior.

Conclusion: This Random Forest model is simple, effective, and can highlight potential insider threats while giving insight into why certain employees are flagged.